

Arkay Solutions

Rule-Based Marketing Automation Agent

Final Consolidated Design Specification

The Specification

One document, sequentially numbered, no stage references. Each section opens with plain-English intent (for Rachel), followed where needed by a fixed procedure block (for the coding agent). Placeholder brackets: none. Go-live checks: two, each with a documented fallback (§12, §19).

1. Product Intent

The agent creates value through subtraction. It removes isolated signup sheets, manual contact harvesting, fragmented reporting, and preventable lead leakage from Arkay Solutions' content-led funnel — without adding new software for anyone to log into.

- It is NOT a live social media manager, a CRM dashboard, or a predictive/black-box AI.
- It IS invisible middleware: rule-based, deterministic, Google-native, editorial in every subscriber-facing touch.

Scope boundary: the system begins at Capture (a contact exists) and ends at outcome logging after the discovery call. Top-of-funnel content operations (LinkedIn discovery posting) are explicitly out of scope for this build.

2. Architecture

Google Apps Script runs all logic. Google Sheets is the operational source of truth. Google Calendar is the booking source. EmailOctopus is the one email platform (audience, sends, engagement webhooks). ZeroBounce verifies emails. Gmail (Workspace) carries internal digests and alerts only.

Layer	Tool	Role
Orchestration	Google Apps Script	All triggers, schedules, API calls, logic.
Source of truth	Google Sheets	Master audience, logs, operational state.
Booking	Google Calendar	Booking detection, digest timing.
Email platform	EmailOctopus	Contacts, tags, automations, campaign sends, engagement webhooks.
Verification	ZeroBounce	4-way email verdicts before the send path.
Internal notifications	Gmail (Workspace acct)	Digests, reports, alerts to Rachel. Transactional only.

Why this stack: it is the cheapest centralized shape — one ESP, no new subscriptions, and every moving part lives in accounts the team already controls.

3. User Model & Permissions

Rachel operates by email only — digests in, decisions out — and never needs a new login. Ali and Devi administer the machinery. Enforcement is by Google account access, since Apps Script has no internal role system.

Role	Who	Access
Operator	Rachel	Receives digests/reports/alerts. No edit access to script project or master Sheet.
Admin	Ali, Devi	Edit access to script project, Sheet, connectors. Hold all API keys (Script Properties). Run imports/maintenance.
Subscriber	Prospects	No system access; sees only branded email.

4. Funnel Coverage

Funnel stage	What the system does
Capture contact	Ingests from Google Form, Luma, Eventbrite, and manual LinkedIn import.
Consolidate	Normalizes to one schema, verifies, dedupes, blocks invalid emails.
Engage	Sends the weekly newsletter and event communications via EmailOctopus.
Book call	Detects Calendar bookings, updates state, schedules the pre-call digest.
Run call / outcome	Delivers Rachel a pre-call digest; captures won/lost afterward.

(Get-Attention is out of scope per §1.)

5. Unified Data Dictionary

Every contact, from every source, is normalized into one typed schema before it is written anywhere. In plain terms: one row shape, no exceptions.

Field	Type	Allowed / format	Default	Required
email	string	RFC-5322, lowercased, trimmed	–	Yes (key)
email_hash	string	MD5 of lowercased email	derived	Yes
first_name	string	title-case (see 5.1)	empty	No
last_name	string	title-case (see 5.1)	empty	No
phone	string	E.164 if present	empty	No
company	string	trimmed free text	empty	No
job_title	string	trimmed free text	empty	No
source	enum	google-form luma eventbrite linkedin-manual calendar-booking	–	Yes
event_name	string	luma/eventbrite only	empty	No
created_at	datetime	ISO-8601 UTC	now()	Yes
verification_status	enum	unverified valid invalid risky	unverified	Yes
lifecycle_state	enum	§7 table (single source of truth)	new	Yes
last_engagement_at	datetime	ISO-8601 UTC or empty	empty	No
last_email_sent_at	datetime	ISO-8601 UTC or empty	empty	No

Field	Type	Allowed / format	Default	Required
tags	string	sorted, semicolon-joined set (§8)	empty	No
archived	boolean	true false	false	Yes
esp_contact_id	string	EmailOctopus member id	empty	No
error_flag	boolean	true false	false	Yes
last_processed_at	datetime	ISO-8601 UTC	now()	Yes

5.1 Normalization rules

- Title-case: capitalize on whitespace and hyphen boundaries; preserve an existing internal capital in the raw input (McLeod stays McLeod, O'Brien stays O'Brien). Apply only to non-empty values.
- Phone: normalize to E.164 when a country can be inferred; otherwise store digits as given. Never reject a contact for a bad phone.
- Tags: a set. Adding an existing tag is a no-op; stored sorted and semicolon-joined; every write is union(existing, incoming).

6. Lifecycle State Model

A contact is always in exactly one state. States move only along the arrows below; anything else is an error the system logs and refuses. In plain terms: no contact can teleport.

State	Entry trigger	Exit → next	Set by
new	Record created from any source	Verification → verified or invalid	Intake
verified	verification_status = valid	ESP sync succeeds → active	Verify
invalid	verification_status = invalid	Manual re-verify only → new	Verify
active	Synced to ESP; eligible for sends	Booking detected → booked	Sync
booked	Calendar booking matched	Outcome → won lost; no-show → active	Booking
won	Rachel logs win	Terminal	Outcome
lost	Rachel logs loss	Optional re-nurture → active	Outcome
stale-60	0 opens/clicks in 60 days (monthly job)	Re-engagement → active; else → archived	Maintenance
archived	Monthly job archives	Manual restore → active	Maintenance

How Rachel logs outcomes: the pre-call digest email contains two mailto links (WON / LOST). Clicking one sends a templated reply the system parses on its 15-minute cycle to set the state. No dashboard, consistent with §1.

7. Tag Taxonomy

One namespace, written identically to the Sheet and EmailOctopus, so tags can never drift from states.

```
source:{google-form|luma|eventbrite|linkedin-manual|calendar-booking}
      (accumulates, never removed)
event:{slugified-event-name}
      (accumulates)
status:{new|verified|invalid|active|booked|won|lost|stale-60|archived}
      (exactly ONE; mirrors lifecycle_state;
      old removed + new added in one call)
flag:{invalid-undeliverable}
      (set on hard bounce / invalid verdict)
send:{newsletter}
      (transient; §12 send mechanism only)
```

ESP status vs our taxonomy — one rule: when a contact goes invalid, the Sheet records lifecycle_state=invalid; in EmailOctopus the contact's platform status is set to unsubscribed (so the ESP itself blocks sends) AND the tag flag:invalid-undeliverable is added. The ESP status field and our tag namespace are different systems; both are set, neither is a substitute for the other.

8. Dedupe & Merge Policy

Identity is the email address, hashed. New data about a known person merges instead of duplicating. When old and new disagree, the rules below decide — no judgment calls in code.

- Key: email_hash (MD5 of lowercased email). Lookup on every intake: found → MERGE, else INSERT.
- Blank incoming never overwrites populated existing.
- Stable fields (email, created_at, source history): existing wins, never overwritten.
- Mutable fields (first_name, last_name, phone, company, job_title): both non-blank and different → incoming (newer) wins; the prior value is written to the Audit log.
- Tags: union, never removed by merge (only the §7 status rule removes a status tag).
- linkedin-manual imports never overwrite any populated core field; they may only add tags.
- invalid contacts stay blocked until the explicit re-verify path returns them to new.

9. Source Ingestion

Each source has one trigger, one mapping, one SLA. All four funnel into the same §5 schema and §8 merge.

9.1 Google Form

```
Trigger: installable onFormSubmit on the linked responses Sheet.
Map by QUESTION TITLE (case-insensitive, trimmed):
  'Email Address'->email 'First Name'->first_name 'Last Name'->last_name
  'Phone'->phone 'Company'->company 'Job Title'->job_title
Unmapped questions ignored. Missing Email column -> row to Error Log
  reason 'missing_email_column'.
Then: verify (§10) -> normalize -> merge (§8) -> Sheet write ->
  EmailOctopus upsert (POST create-contact; on MEMBER_EXISTS -> PUT
  update) -> tag source:google-form.
SLA: median < 60s submit-to-write; p95 < 120s (trailing 7 days).
```

9.2 Eventbrite

```
Trigger: Eventbrite webhook -> published Apps Script Web App doPost.
Same pipeline; tags source:eventbrite + event:<slug>.
SLA: < 5 min.
```

9.3 Luma

```
Method: scheduled CSV into Drive folder Arkay/Luma-Inbox (Luma export).
Trigger: time-driven poll every 15 min; cursor = file id + modifiedTime
  in Script Properties.
Parse -> same pipeline -> tags source:luma + event:<slug> ->
  move file to Arkay/Luma-Done (never re-ingested; idempotent).
SLA: < 60 min.
```

9.4 LinkedIn manual import

```
Trigger: Admin runs import script; picks CSV/XLSX from Drive.
Clean: strip emojis/control chars, fix casing, trim.
Match by email_hash; merge TAGS ONLY (never core fields, §8).
```

Tag source:linkedin-manual. Verification still runs (§10).
No scraping, ever (§17).

10. Verification & Filtration

Every new email address is checked before it can receive anything. Real verifiers return four verdicts, not two, and the system handles all four — including what happens when the verifier itself is down.

```
Provider: ZeroBounce single-email API via UrlFetchApp.
Key: Script Properties -> ZEROBOUNCE_API_KEY. Free tier ~100/mo;
    $14 cap watcher alerts Rachel at 90% usage.
Called on: first sight of an email, and the re-verify paths (§15).
Verdict map -> verification_status:
    valid          -> valid    (proceed to sync + sends)
    invalid        -> invalid  (lifecycle_state=invalid;
                                ESP status=unsubscribed;
                                tag flag:invalid-undeliverable;
                                silent log, no alert)
    catch-all/unknown -> risky (sync allowed; bulk sends suppressed;
                                joins next re-verify batch)
    do-not-mail/abuse -> invalid
Timeout 10s. On timeout / 5xx / quota: set risky, queue for retry
next run. NEVER hard-block a contact because the verifier failed.
```

11. Booking Detection & Pre-Call Digest

When someone books a discovery call on Rachel's calendar, the system notices within minutes, updates their state, and makes sure Rachel has a one-page digest in her inbox at least an hour before the call — including reschedules.

```
Calendar: Rachel's primary ('primary' when the script runs as Rachel;
    else CALENDAR_ID in Script Properties).
Detection: installable calendar onEventUpdated trigger -> fetch changed
    events (the trigger payload carries no detail).
A booking = ALL of:
    - >=1 guest whose email domain != the Arkay domain
    - title matches /discovery|call/i
    - status == 'confirmed'
Attendee = first external guest (alphabetical if several) ->
    email_hash -> contact match.
    No match -> minimal contact created (source:calendar-booking),
    logged 'booking_no_contact_match'; digest still sends.
On booking: lifecycle_state=booked; status:booked (one atomic call).
Digest timing: Calendar can't fire at 'T-1h', so a 15-min time-driven
    scan finds events starting in 60-75 min not yet digested ->
    send digest -> mark digested (event id in Properties).
Reschedule: time change clears the digested mark -> re-digests.
Cancel/decline: lifecycle_state=active; status:active.
```

Digest contents: name, email, company/title if known, source, tags, last 3 emails sent with open/click status, and the WON / LOST mailto links (§6).

12. Weekly Newsletter

Every Thursday morning the system checks the blog. New post → a branded newsletter goes to the active audience with zero manual steps. No new post → it skips quietly. The same post can never be sent twice.

One-time EmailOctopus setup (Admin, before go-live):

1. Create audience custom fields NEWSLETTER_HTML and NEWSLETTER_SUBJECT (text).
2. Build one automation Weekly Newsletter whose trigger is **tag added:** send:newsletter. Single email step: subject {{NEWSLETTER_SUBJECT}}, body {{NEWSLETTER_HTML}}. Activate it.
3. Store its id as EO_NEWSLETTER_AUTOMATION_ID in Script Properties.

Why tag-triggered, not API-triggered: EmailOctopus's “manually via API” trigger fires only once per subscriber, ever — useless for a weekly loop. A tag-added trigger re-fires every time the tag is re-added, which is exactly the weekly behavior needed. This is the reconciled mechanism.

```
Weekly trigger: Thursday 09:00 America/Chicago (project TZ matches).
1. Fetch RSS_URL (default <blog-root>/feed; see §19 go-live check).
   Parse with XmlService: item[0] title, link, guid, pubDate, summary.
2. pubDate older than 7 days -> log 'Newsletter Skipped'; exit.
3. Idempotency: Script Property newsletter_sent_for == guid -> exit.
4. Build branded HTML (template §17) from the post fields.
5. Segment = lifecycle_state active AND verification valid AND NOT
   stale/archived.
6. For the segment, batched under 10 req/s with a resume cursor:
   PUT update-contact: set NEWSLETTER_HTML + NEWSLETTER_SUBJECT,
   then add tag send:newsletter -> automation fires per contact.
7. After confirmed entry: remove send:newsletter (ready for next week).
8. Success: set newsletter_sent_for=guid; success notice to Rachel.
   Failure after 3 retries (2s/4s/8s): guid NOT set; error logged;
   alert per §14; next Thursday retries the same post.
FALLBACK (go-live check §19 only): if tag-triggered entry proves
non-functional, EO_SEND_MODE=manual -> Apps Script emails Rachel the
built HTML + one-click send link. Documented, coded, off by default.
```

13. Engagement, Webhooks & Reporting

Opens, clicks, bounces, and unsubscribes flow back from EmailOctopus in real time; every Monday Rachel gets one email with the numbers that matter. Nothing to log into.

```
Receiver: Apps Script Web App doPost, deployed execute-as Admin,
access 'Anyone'; URL registered in EmailOctopus.
Auth: HMAC-SHA256 over the raw body vs EO_WEBHOOK_SECRET
(Script Properties). Bad signature -> drop + log.
Batching: up to 1000 events/request; iterate all.
Idempotency: dedupe on event id (rolling processed-id set).
Effects: opened/clicked -> last_engagement_at=now.
bounced -> verification_status=invalid + §10 invalid handling.
unsubscribed -> ESP handles suppression; Sheet mirrors status.
Weekly report (Mon 08:00 America/Chicago): campaign report endpoints
for trailing 7 days -> Reporting sheet -> one branded email:
open rate, click rate, event attendance, audience growth,
stale count, source trends.
Metric formulas: duplicate_rate = dup_email_rows/total_rows;
validation_coverage = rows(valid|invalid)/total_rows;
open_rate = unique_opens/delivered per campaign, 7-day window.
```

14. Failure Handling & Cap Watcher

Failures are logged, retried, and surfaced — never silent, never fatal. And before any free tier runs out, Rachel gets a plain-language heads-up so nothing breaks and nothing bills without her say-so.

- Error Log sheet: ts_utc, source, event_type, reason, payload_ref, retry_status, resolution_status.
- Transient errors (5xx/timeout/quota): 3 retries, exponential backoff (2s/4s/8s), then dead-letter row + next-run pickup.
- Illegal state transition: rejected, logged, contact untouched.
- Batched failure digest to Rachel within 24h of first failure; pipeline-down (webhook receiver erroring, ESP unreachable) alerts immediately.

- Cap watcher (daily): EmailOctopus contacts $\geq 90\%$ of 2,500 $\rightarrow$ alert; ZeroBounce credits $\geq 90\%$ of monthly $\rightarrow$ alert. Alert = usage, ceiling, next-tier price, “reply UPGRADE to proceed.” No auto-upgrades, ever.
- Long jobs (imports, maintenance, newsletter field-writes) batch under the 6-minute limit with resumable cursors in Script Properties.

15. Monthly Maintenance

Once a month the system quietly cleans house: finds subscribers who've gone dark for 60 days, double-checks their addresses, archives the truly gone, and tells Rachel what it did.

```
Trigger: 1st of month, 02:00 America/Chicago.
stale-60 = last_engagement_at older than 60d (or null AND created_at
older than 60d).
Steps: tag status:stale-60 -> re-verify (§10) -> still no engagement
AND not valid -> archive: archived=true, status:archived,
ESP status=unsubscribed (drops from billable count; record kept).
Never hard-delete.
Batched under the 6-min limit; resume cursor.
Summary email to Rachel: archived count, reasons, live list size.
```

16. Guardrails

- No LinkedIn scraping. Manual CSV/XLSX import is the only LinkedIn path.
- All keys in Apps Script PropertiesService (Script Properties). Never in code, never in the Sheet.
- SPF, DKIM, DMARC configured on the Arkay domain before production-scale sending. Hard gate.
- Rate limits respected by design: EmailOctopus 10 req/s (batched writes), Apps Script URLFetch/MailApp daily quotas (§18).
- Transactional email only through the Workspace account (~1,500/day MailApp ceiling); the newsletter never goes through MailApp.

17. Brand & Design System

Everything the system sends looks like The Angle wrote it: warm, editorial, unmistakably Arkay — and it must survive real inboxes, including dark mode.

- Headings: Garamond/EB Garamond $\rightarrow$ fallback Georgia, 'Times New Roman', serif.
- Body: Georgia/Inter $\rightarrow$ fallback Georgia, Arial, sans-serif.
- Labels/metadata: Space Mono/JetBrains Mono $\rightarrow$ fallback 'Courier New', monospace, ALL-CAPS, tracked out.
- Palette: canvas #F4F1EA, card #FAF9F6, text #1A1A1A, accent #C27D53, grid #D1CDC4.
- Contrast rule: body text stays on #1A1A1A; the copper accent is for large labels/tags only (fails WCAG at small sizes).
- One branded HTML template with a plaintext alternative part; every send uses it; QA verifies in Gmail and Apple Mail, light and dark mode.

18. Apps Script Runtime Constraints

Constraint	Design consequence
6-minute execution limit	All bulk jobs batch with resumable cursors (already required in §12, §15).
MailApp/GmailApp cap (~1,500/day Workspace)	Transactional only; newsletter goes through EmailOctopus, never MailApp.
URLFetchApp daily quota	Verification + ESP calls batched and cached to stay under cap.

Constraint	Design consequence
Trigger types	Installable triggers only (onFormSubmit, calendar, time-driven); simple triggers cannot call external APIs or send mail.
Secrets	Script Properties (no Secret Manager in vanilla Apps Script).
Webhook endpoints	One deployed Web App (doPost), 'Anyone' access + HMAC verification (§13).

19. Go-Live Checks (the only two open facts)

Two facts about external systems can only be confirmed against live accounts. Both have working fallbacks already specified, so neither blocks the build — they decide configuration, not design.

Check	How verified	If it fails
EmailOctopus tag-added trigger enters contacts into the Weekly Newsletter automation repeatedly (weekly re-entry)	3-call live test on the real account: create contact w/ tag → add send:newsletter → confirm automation entry; repeat once to prove re-entry	EO_SEND_MODE=manual (§12 fallback: one-click send email to Rachel). Build unaffected.
Blog RSS endpoint path	Fetch <blog-root>/feed once at go-live	Set RSS_URL Script Property to the confirmed path; parsing logic unchanged.

20. Build Sequence & Acceptance

Build in this order — each step is testable before the next begins:

4. Sheet structure + schema + merge functions (unit-testable in isolation).
5. Script Properties + secrets + timezone config.
6. Google Form intake → first end-to-end row.
7. Verification (ZeroBounce) + invalid handling.
8. EmailOctopus upsert + tag sync.
9. Eventbrite webhook + Luma folder poll.
10. LinkedIn import script.
11. Calendar detection + digest scheduler + outcome mailtos.
12. Newsletter (incl. the §19 live test, first).
13. Webhook receiver + weekly report.
14. Monthly maintenance + cap watcher.
15. End-to-end QA against the acceptance table.

Acceptance criteria (all computable):

Criterion	Testable definition
Duplicate rate < 2%	$\text{duplicate_email_rows} / \text{total_rows} < 0.02$, full sheet
Validation coverage ≥ 90%	$\text{rows with status in \{valid, invalid\}} / \text{total_rows} \geq 0.90$
All 4 sources land correctly	5 spot-checks per source show correct schema + source tag
Form capture SLA	median < 60s, p95 < 120s over 7 days

Criterion	Testable definition
Pre-call digest timing	<code>digest_sent ≤ event_start - 60 min</code> , 100% of sampled bookings
Newsletter automation	new post → sent, no manual steps; no-post week → logged skip; same guid never sent twice
Reports arrive	weekly + monthly emails on schedule, 2 consecutive cycles
Brand fidelity	digest matches §17 in Gmail + Apple Mail, light + dark
Zero new subscriptions	stack = Google + EmailOctopus + ZeroBounce only

End of specification. *One document, no stage references, no [CONFIRM] brackets; two go-live checks with coded fallbacks. Build starts at §20 step 1.*